

Object Oriented Systems (OOS) — CO3 Detailed Solution



Generic Class with Binary Search Using Different Datatypes

Based on the uploaded 2021 examination paper.



Introduction

Modern programming languages are expected to support:

-  reusability
-  flexibility
-  type safety
-  efficient searching mechanisms

Java achieves these capabilities through two extremely powerful concepts:  **Generics + Binary Search**

In this question, we must create:

-  a generic class
-  a generic member method `find(A,P)`
-  support for multiple datatypes
-  efficient binary searching
-  position detection of the searched element

The solution demonstrates both:

-  **Generic Programming**
-  **Searching Algorithms**

which are fundamental concepts in modern enterprise Java systems.  



Exact Question

Define a generic class with a member method `find(A,P)` that takes sorted arrays `A` of different datatypes (such as `int`, `float`, `char`) as input and checks whether the element `P` is present (and at what position) within the array or not using binary search technique.



Introduction to Generic Programming

Before Java Generics were introduced, programmers used `Object` type references to create reusable code. However, this created several serious problems:

-  explicit type casting
-  runtime errors

-  `ClassCastException`
-  poor readability

To solve these issues, Java introduced  **Generics** in Java 5.

What Are Generics?

Generics allow:

-  datatype-independent programming
-  reusable algorithms
-  compile-time type checking
-  elimination of unnecessary casting

Instead of writing separate programs for integers, floats, and characters, a single generic implementation handles everything elegantly.

Generic Class Syntax

```
class Demo<T> {  
}
```

Here, **T** → **Type Parameter**. **T** can represent: `Integer`, `Float`, `Character`, `Double`, `String`, or Custom Objects.

Introduction to Binary Search

Binary Search is one of the most efficient searching algorithms. Instead of checking every element sequentially:

-  it repeatedly divides the search space
-  eliminates half of remaining elements
-  performs searching logarithmically

Time Complexity of Binary Search

Case	Complexity
Best Case	$O(1)$
Average Case	$O(\log n)$
Worst Case	$O(\log n)$

Why Binary Search Is Fast

Suppose array size is `1024`.

- Linear search may require **1024 comparisons**.

- Binary search requires only **10 comparisons** because $2^{10} = 1024$.

Thus binary search is extremely efficient for large sorted datasets. 🔥

📁 Core Concept of This Problem

We must:

- ✅ define generic class
- ✅ define method `find(A,P)`
- ✅ support multiple datatypes
- ✅ perform binary search
- ✅ display position if found

🚀 Important Generic Constraint

For binary search, elements must be comparable. Thus the generic type must implement `Comparable`. Otherwise, comparison becomes impossible.

📁 Generic Class Definition

```
class Search<T extends Comparable<T>>
```

means:

- ✅ `T` can be any datatype
- ✅ but must support comparison operations

💻 Complete Java Program

```
class Search<T extends Comparable<T>> {  
  
    void find(T arr[], T key) {  
        int low = 0;  
        int high = arr.length - 1;  
        boolean found = false;  
  
        while(low <= high) {  
            int mid = (low + high) / 2;  
            int result = key.compareTo(arr[mid]);  
  
            if(result == 0) {  
                System.out.println("Element Found at Position: " + mid);  
                found = true;  
                break;  
            } else if(result > 0) {  
                low = mid + 1;  
            }  
        }  
    }  
}
```

```
        } else {
            high = mid - 1;
        }
    }

    if(!found) {
        System.out.println("Element Not Found");
    }
}

class Demo {
    public static void main(String args[]) {
        // Integer Search
        Search<Integer> obj1 = new Search<>();
        Integer arr1[] = {10, 20, 30, 40, 50, 60};
        obj1.find(arr1, 40);

        // Float Search
        Search<Float> obj2 = new Search<>();
        Float arr2[] = {1.1f, 2.2f, 3.3f, 4.4f, 5.5f};
        obj2.find(arr2, 3.3f);

        // Character Search
        Search<Character> obj3 = new Search<>();
        Character arr3[] = {'a', 'c', 'e', 'g', 'i'};
        obj3.find(arr3, 'g');
    }
}
```

Output

```
Element Found at Position: 3
Element Found at Position: 2
Element Found at Position: 3
```

Why This Output Occurs

Integer Search

Array: 10 20 30 40 50 60 Searching: 40

Step-by-Step Binary Search

Step	low	high	mid	arr[mid]
1	0	5	2	30
2	3	5	4	50

Step	low	high	mid	arr[mid]
3	3	3	3	40

Element found at: `index = 3`



Float Search

Array: `1.1 2.2 3.3 4.4 5.5` Searching: `3.3` Found at: `index = 2`



Character Search

Array: `a c e g i` Searching: `g` Found at: `index = 3`



Deep Conceptual Analysis



Why Comparable Interface Is Necessary

Inside binary search, `key.compareTo(arr[mid])` is used for comparison. Without `Comparable`,  generic objects cannot be compared.



Meaning of compareTo()

Return Value	Meaning
<code>0</code>	Equal
<code>Positive</code>	Greater
<code>Negative</code>	Smaller



JVM Internal Behavior

During compilation, `Search<Integer>` and `Search<Float>` undergo  **Type Erasure**. The compiler internally converts them into `Search<Object>` while preserving compile-time type safety. Thus:

-  backward compatibility maintained
-  generic behavior achieved



How Binary Search Works Internally

Binary search repeatedly performs a middle comparison and removes half of the search space. Example: `[10 20 30 40 50 60]` Searching for `40`: `mid → 30` Since `40 > 30`, the left half is discarded. This divide-and-conquer strategy makes searching extremely fast.



Real-World Intuition Imagine searching a word in a dictionary.  You do NOT start from the first page. Instead:

-  open middle page
-  decide direction
-  repeatedly divide pages

Binary search works exactly the same way.

Advantages and Limitations

Advantages of Generic Binary Search

Advantage	Explanation
Reusable	Same code works for many datatypes
Efficient	$O(\log n)$ complexity
Type Safe	Compile-time checking
Cleaner Code	No duplicate searching functions
Maintainable	Single implementation

Limitations

Limitation	Explanation
Requires Sorted Array	Unsorted arrays fail
Wrapper Classes Needed	Primitive types not directly allowed
Comparable Dependency	Objects must support comparison

Primitive vs Wrapper Class Discussion

Java Generics work only with objects. Thus wrapper classes are used:

Primitive	Wrapper
<code>int</code>	<code>Integer</code>
<code>float</code>	<code>Float</code>
<code>char</code>	<code>Character</code>

Alternative Without Generics

Without Generics, separate methods are required: `findInt()`, `findFloat()`, `findChar()`. This causes:

-  code duplication
-  poor maintainability

Generics elegantly solve this issue.

Real-World Applications

Generic Binary Search is used in:

-  database indexing

-  searching libraries
 -  file systems
 -  compiler symbol tables
 -  collection frameworks
 -  AI search systems
-

Final Conclusion

This problem beautifully combines:  **Generic Programming + Binary Search**

The solution demonstrates how Java supports reusable algorithms, efficient searching, type-safe programming, and compile-time flexibility. By using Generics and Binary Search together, developers can create scalable and efficient software systems capable of handling multiple datatypes with minimal code duplication.   